

Project 2 code report

Overview:

For my code it implements the producer-consumer problem. With the use of threads and synchronization primitives, the code will run multiple threads a bounded buffer.

The code defines a circular buffer with a maximum size that is used by the producer thread and two consumer threads. The producer will create a number of items and put them into the buffer, while the consumers remove and process them. To make sure the producer does not use buffer and that the consumers do not read from an empty buffer i used two semaphores, one counting open slots and one that counts open items.

The mutex is used to guard the critical section where the buffer is accessed or modified. This will make sure that one thread can use a shared buffer at a time.

The producer ran in a loop. It waits for an empty slot then inserts an item. After that it opens the buffer and signals that the new item is available. The consumers have a similar loop where it instead removes an item then signals that the slot is empty.

Test case:

```
[producer 1] starting
[producer 1] Waiting
[producer 1] Waiting
[producer 1] produced item 0 at index 0
[producer 1] release after
[producer 1] Waiting
[producer 1] Waiting
[producer 1] produced item 1 at index 1
[producer 1] release after
[consumer 2] starting
[consumer 2] waiting for item
[consumer 2] waiting for lock
[consumer 2] got item 0 from index 0
[consumer 2] released lock after getting
[consumer 3] starting
[consumer 3] waiting for item
[consumer 3] waiting for lock
[consumer 3] got item 1 from index 1
[consumer 3] released lock after getting
[producer 1] Waiting
[producer 1] Waiting
[producer 1] produced item 2 at index 2
[producer 1] release after
[consumer 2] waiting for item
[consumer 2] waiting for lock
[consumer 2] got item 2 from index 2
[consumer 2] released lock after getting
[consumer 3] waiting for item
[producer 1] Waiting
[producer 1] Waiting
[producer 1] produced item 3 at index 3
[producer 1] release after
[consumer 3] waiting for lock
[consumer 3] got item 3 from index 3
[consumer 3] released lock after getting
[consumer 2] waiting for item
[producer 1] Waiting
[producer 1] Waiting
[producer 1] produced item 4 at index 4
[producer 1] release after
[consumer 3] waiting for item
[consumer 3] waiting for lock
[consumer 3] got item 4 from index 4
[consumer 3] released lock after getting
[producer 1] Waiting
[producer 1] Waiting
```

```
[producer 1] produced item 5 at index 0
[consumer 3] finished.
[producer 1] release after
[consumer 2] waiting for lock
[consumer 2] got item 5 from index 0
[consumer 2] released lock after getting
[producer 1] finished.
[consumer 2] finished.
program done.
```